

Braitenberg Vehicles in a Browser Simulator

1. How the drop-down choice reaches `loadVehicle()`

In `BraitenbergABCD.html` the drop-down is a `<select id="vehicleSelectBox">` with the options A, B, C and D. The select element itself has no event handler, so choosing an option does nothing yet. Next to it is a *Load* button with `onclick="javascript:loadVehicle();" , which calls the function without any argument.`

The choice is therefore not passed as a parameter. Instead, `loadVehicle()` reads it from the page (the DOM) when it runs:

```
switch(document.getElementById("vehicleSelectBox").selectedIndex) { ... }
```

`selectedIndex` is the zero-based position of the selected option, so $A = 0$, $B = 1$, $C = 2$ and $D = 3$. The `switch` statement uses this number to pick the configuration. In the original file only `case 0` existed; every other index fell through to the `default` branch, which opens a prompt where the vehicle's JSON description can be edited by hand. We kept that branch as a fallback.

The new settings are written into `simulationInfo.bayVehicle`, which is the vehicle currently shown in the "bay" at the bottom left. This is the last vehicle that was clicked or dragged (vehicle 1 by default). After the switch, `loadBay()` redraws the bay and updates the sensor controls. So when there are several vehicles in the arena, you first click the one you want to change and then press Load.

2. Manipulating the sensor locations

A sensor's `position` is an angle measured from the front of the vehicle. Because the y-axis of the HTML canvas points downwards, a negative angle lies on the vehicle's *left* side and a positive angle on its right side. The sensors sit on the rim of the body (radius 15 px), so the sideways distance between two sensors at $\pm\theta$ is $2 \cdot 15 \cdot \sin \theta$. This is about 21 px at $\pm\pi/4$ but only 5.9 px at $\pm\pi/16$.

The sensors are not directional. Each one simply measures the light intensity at its own position, and that intensity falls off with distance to the bulb (roughly as $1/\text{distance}$ in the simulator). The only reason the left and right sensor read different values is that one of them is a bit closer to the light. The vehicle's own body also casts a small shadow.

Prediction. If the sensors are moved closer together ($\pm\pi/16$), both sensors are at almost the same distance from the light. The difference between the two motor signals, which is what makes the vehicle turn, becomes much smaller. We expected the vehicle to keep the same type of behaviour (the wiring does not change) but to steer much more weakly. It would drive in straighter lines and react later, and it could miss a light that is off to the side.

Observations. We tested this with the default (aggression) vehicle, both by dragging the sensors in the bay and by recording trajectories. In the controlled test the vehicle starts driving straight ahead with the light 120 px to the side of its path (Figure 1, Table 1).

Sensor positions	Sideways separation	Heading change in first 15 steps	Steps until the light is reached
$\pm\pi/2$	30 px	30°	114
$\pm\pi/4$ (default)	21 px	5°	81
$\pm\pi/8$	11 px	2°	115
$\pm\pi/16$	5.9 px	1°	195 (misses the first time, circles back)
$\pm\pi/32$	2.9 px	0°	not reached within 260 steps (circles around it)

Table 1. Aggression vehicle with the light 120 px to the side of its initial path.

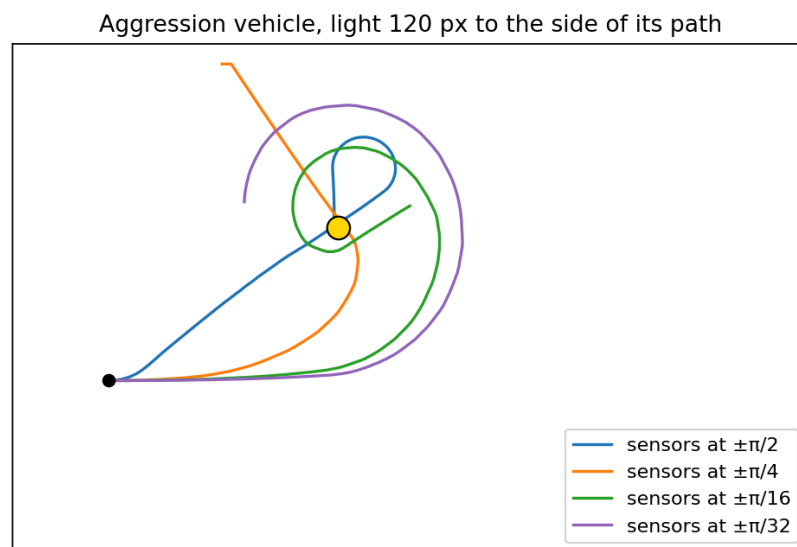


Figure 1. Paths of the aggression vehicle for different sensor positions. The black dot is the start position.

This matched our prediction. With the sensors at $\pm\pi/16$ the vehicle hardly turned at first, drove past the light, and only reached it after a wide loop. At $\pm\pi/32$ it kept circling around the light without hitting it. Placing the sensors far apart ($\pm\pi/2$) gave the strongest steering; the vehicle turned sharply towards the light right away. It was not faster than the default placement, though, because the hard turns cost forward progress, and after passing the light it looped back to it again. Two more things we noticed while dragging sensors:

- Sensors near the back of the vehicle ($\pm 3\pi/4$) still work, since the steering only depends on which sensor is closer to the light.
- If the two sensors are dragged past each other (the "left" sensor ends up on the right side), the crossed wires effectively become uncrossed. The aggression vehicle then behaves exactly like the fear vehicle; in our recordings the trajectories were identical.

3. Implementing the four vehicles

The default vehicle has its left sensor ($-\pi/4$) connected to the *right* motor and its right sensor ($+\pi/4$) to the *left* motor, with a positive (excitatory) strength of 1. When the light is on its left, the right

wheel turns faster, so the vehicle turns left, towards the light, and speeds up as it gets closer. This is Braitenberg's vehicle **2b, aggression** (the code comment also labels it this way). It drives straight into the light at full speed.

In the code, a sensor's `type` is the motor it drives ("left" or "right"), not the side it is on. The sign of `strength` sets whether the connection is excitatory (+) or inhibitory (−), and `baseValues` gives the motor speeds when there is no light. We added cases 1–3 to `loadVehicle()` in the same style as case 0. We also gave each type its own colour so they can be told apart with several vehicles in the arena.

Option	Vehicle	Wiring	Connection	Base speed	Strength	Colour
A	Aggression (2b)	crossed	excitatory	1	+1	red
B	Love (3a)	uncrossed	inhibitory	3	−0.5	magenta
C	Exploration (3b)	crossed	inhibitory	3	−0.5	green
D	Fear (2a)	uncrossed	excitatory	1	+1	blue

Table 2. Architectures implemented in `loadVehicle()`. All sensors sit at $\pm\pi/4$. "Crossed" means the left sensor drives the right motor and vice versa.

For example, the love vehicle is implemented as:

```
case 1: //Love (3a): uncrossed, inhibitory
simulationInfo.bayVehicle.vehicle.color = "magenta";
simulationInfo.bayVehicle.vehicle.baseValues = {left: 3, right: 3, bulb: 0};
simulationInfo.bayVehicle.vehicle.sensors = [
  {position: -Math.PI/4, type: "left", strength: -0.5},
  {position: Math.PI/4, type: "right", strength: -0.5}];
break;
```

Choosing the parameters of the inhibitory vehicles

Fear only needed the wires uncrossed. For love and exploration our first attempt was to copy the default values and flip the sign (base speed 1, strength −1). That did not work: the vehicle immediately drove *backwards*, away from the light. Reading `moveVehicle()` explained why. Each sensor adds `strength × light × bulbPower` to its motor, and `bulbPower` (5) is applied twice (also in `getSensorResponse()`), so the effective gain is 25. Near a single bulb the light value is about $10/d$ (d = distance in pixels, capped at 1). At 200 px this already gives an inhibition of 1.25, which is more than the base speed of 1, so the motor speeds became negative.

An inhibitory vehicle can only slow down, so it needs a base speed that is larger than the inhibition it gets far from the light. The love vehicle comes to rest where the inhibition equals the base speed, i.e. at a sensor distance of $d = 250 \cdot |\text{strength}| / \text{base}$. With base 3 and strength −0.5 this predicts 42 px from sensor to bulb. The sensors are about 11 px in front of the vehicle's centre, which gives roughly 53 px centre to centre, close to the 50 px we measured. We also tried other settings (base 1 or 2 with strength −0.5 stopped at 134 px and 71 px), and those matched the formula too. We chose base 3 / −0.5 because the vehicle then stops close to the light but does not touch it, and moves at a speed comparable to the other vehicles. We used the same values for exploration.

Observed behaviour

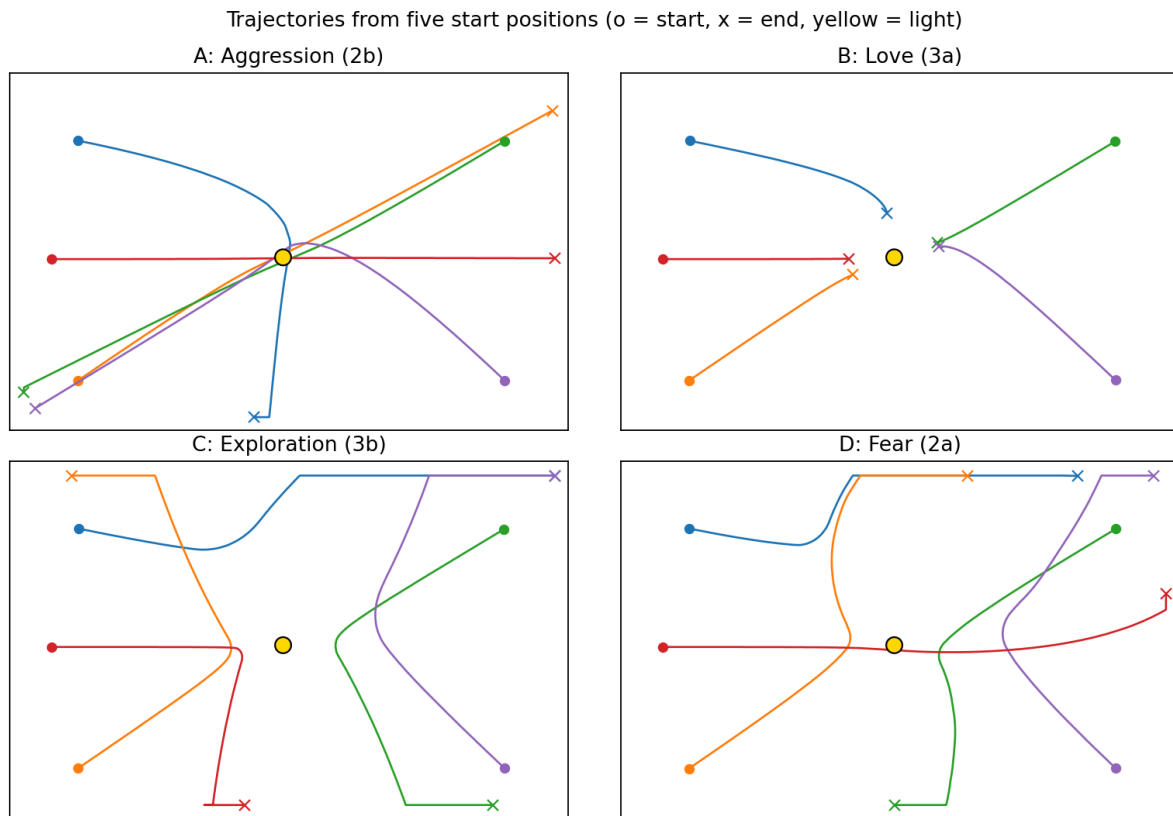


Figure 2. Paths of the four vehicles from the same five start positions with one light. The red start in each panel points straight at the light.

- **A, Aggression:** turns towards the light and speeds up, hitting it at maximum speed from every start position. The light is not a physical obstacle in the simulator, so the vehicle drives through it and then carries on to the wall.
- **B, Love:** turns towards the light while slowing down, and comes to a complete stop about 50 px from it, facing it. It stays there until it or the light is moved.
- **C, Exploration:** slows down as it approaches the light and turns away from it (a heading change of 50–90°), then speeds up again and drives off. In theory it should then go on to "explore" other lights, but in this arena it usually ends up against a wall. Even with three lights it was pushed away from all of them and stayed at the edge of the arena.
- **D, Fear:** turns away from the light and speeds up the closer it is, so it moves away quickly. One exception (red path): when it drives straight at the light, both sensors receive the same amount of light, so there is nothing to turn it and it rushes through the light at full speed. This is an unstable balance; with even a small offset it swerves away.

4. Notes

Things that tripped us up.

- *Left or right?* It took some time to figure out which sensor is the left one. Because the canvas y-axis points down, the negative angle is on the left. The colours in the bay helped us confirm it: a

black sensor drives the right motor and a white one drives the left motor.

- *Naming.* The field `type` names the motor a sensor is connected to, not where the sensor is. Once we understood that, crossed versus uncrossed wiring was just a matter of swapping `"left"` and `"right"`.
- *Inhibitory vehicles.* Flipping the sign of the strength was not enough; the vehicles drove backwards (Section 3). We had to work out the scaling in `moveVehicle()` to choose sensible base speeds.
- *Load only changes one vehicle.* With several vehicles, pressing Load only changes the vehicle in the bay, so each vehicle has to be clicked first.
- *Walls.* The arena clamps the vehicle's position, so vehicles that leave the light (aggression, fear, exploration) end up pushing against a wall forever. For the exploration vehicle this hides its intended behaviour, because it cannot turn around and find the next light. This is a limit of the simulator rather than of the vehicles.
- *Measuring behaviour.* Judging "how fast it turns" by eye was hard, so besides using the web page we ran the simulator's own `moveVehicle()` code in Node.js and recorded the vehicle positions. The figures and numbers in this write-up come from those recordings.

What we found interesting. It is striking how different the behaviours look even though the vehicles differ only in two things: whether the wires cross and the sign of the connection. Without knowing the wiring, it is tempting to describe the vehicles as "angry", "shy" or "in love", while they are just two sensors and two motors. The sensor experiment showed the same idea from another angle: moving the sensors across the midline turns aggression into fear without touching any code. The fear vehicle driving head-on into the light was also a nice example of how a purely reactive system can end up in a situation where its behaviour looks like the opposite of its "character".

What we found difficult. The motion code in `moveVehicle()` (turning along a circle defined by the two wheel speeds) and the lighting code were hard to read at first, and we needed them to understand why our first love and exploration vehicles drove backwards. Once we derived the stopping-distance formula and saw it match the measurements, tuning the parameters became easy.